

Rapport Fintrack – Niveau Intermédiaire

1- Modèle incrémental vs table

Un modèle incrémental est pertinent lorsqu'une table contient un volume important de données. Il permet de ne traiter que les nouvelles lignes ou les lignes modifiées depuis le dernier chargement. Dans FinTrack, `fct_transactions` et `fct_virements` utilisent une stratégie incrémentale `merge`.

Cette approche réduit le temps d'exécution et la consommation de ressources. Elle est particulièrement utile pour les traitements quotidiens de transactions financières. Les principaux pièges sont les doublons, les retards de données et les mises à jour mal détectées. Il faut également définir une clé unique fiable et prévoir les changements de structure.

Un `full-refresh` peut être nécessaire si l'historique a été corrigé ou si la logique du modèle change. Dans FinTrack, la requête incrémentale doit aussi filtrer efficacement les nouvelles données.

Actuellement, les modèles utilisent `merge`, mais relisent encore l'ensemble de leur source. Un filtre basé sur une `date de chargement` ou une `date de transaction` améliorerait donc réellement les performances.

2- SCD (Slow Changing Dimension)

Un SCD Type 2 conserve l'historique des différentes versions d'une même entité. Au lieu d'écraser une ancienne valeur, il crée une nouvelle ligne lorsqu'un attribut change. Chaque version possède généralement une date de début, une date de fin et un indicateur de version courante.

Dans FinTrack, le snapshot `snap_comptes` applique cette logique aux comptes bancaires. Il détecte les changements sur le statut, l'email et le type de compte. Grâce à cet historique, on peut savoir quelles informations étaient valides à une date donnée et pouvoir plus facilement suivre l'historique des opérations en fonction des changements sur le compte. Cela facilite l'audit, l'analyse des comptes et la compréhension des évolutions clients.

3- Macros DBT

Une macro permet de définir une logique SQL ou Jinja réutilisable un peu comme le fait une fonction dans un langage de programmation. Dans le projet, les macros gèrent notamment le `nommage des schémas`, le `suivi des traitements`, les `tests` et les `hooks`. Elles améliorent la maintenance, la cohérence et la réduction des erreurs. La configuration multi-environnement sépare les développements, les tests et la production. Dans le projet, la macro `generate_schema_name` permet de réécrire la logique de nommage des schémas cibles pour éviter le fonctionnement par défaut de DBT lors des déploiements `prod`.

4- Configuration multi-environnement

Les environnements hors production utilisent pour les matérialisations des schémas préfixés par leur schémas cibles, cela limite le risque d'écraser les données de production